Slide Sebelumnya	←
Mode Layar Penuh (Fullscreen)	F
Speaker Notes View (Catatan Dosen)	S
Slide Overview (Matriks)	0 / Esc
Blackout / Layar Hitam (Fokus Diskusi)	B / .

Jaringan Komputer

Pertemuan 04 • Lapisan Data Link

DOSEN PENGAMPU

Rajif Agung Yunmar, S.Kom., M.Cs.

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Hartanto Tantriawan, S.Kom., M.Kom.

Ilham Firman Ashari, S.Kom., M.T.

Program Studi Teknik Informatika · Fakultas Teknologi Industri · Institut Teknologi Sumatera (ITERA)

Outline Materi

Bagian 1: Layanan & Error Handling

- Layanan Lapisan Data Link
- Framing & Struktur Frame
- Deteksi Error: Parity, Checksum, CRC
- Kontrol Aliran & Koreksi Error

Bagian 2: MAC & Jaringan LAN

- Multiple Access Protocols (ALOHA, CSMA)
- MAC Address & ARP
- Ethernet & Frame Ethernet
- Switch: Self-Learning & Forwarding
- Switch vs Router

Analogi: Data Link di Kehidupan Nyata

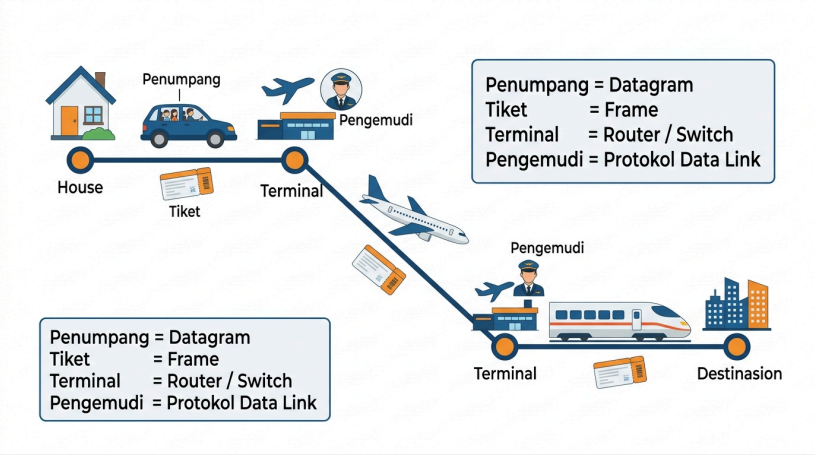
Bayangkan perjalanan **Denpasar → Bandung**:

- 🚖 **Limo**: Denpasar → Ngurah Rai (Bandara)
- ✈️ **Pesawat**: Ngurah Rai → Soetta (Jakarta)
- 🚄 **Kereta**: Jakarta → Bandung

Analogi Jaringan:

- 👤 Penumpang = **Datagram** (paket data)
- 🎫 Tiket per segmen = **Frame** per link
- 🚉 Terminal transportasi = **Router / Switch**
- Pengemudi/pilot = **Link-layer protocol**

Tiap "kendaraan" yang berbeda = tiap link bisa punya protokol berbeda (Ethernet, Wi-Fi, PPP, dll).



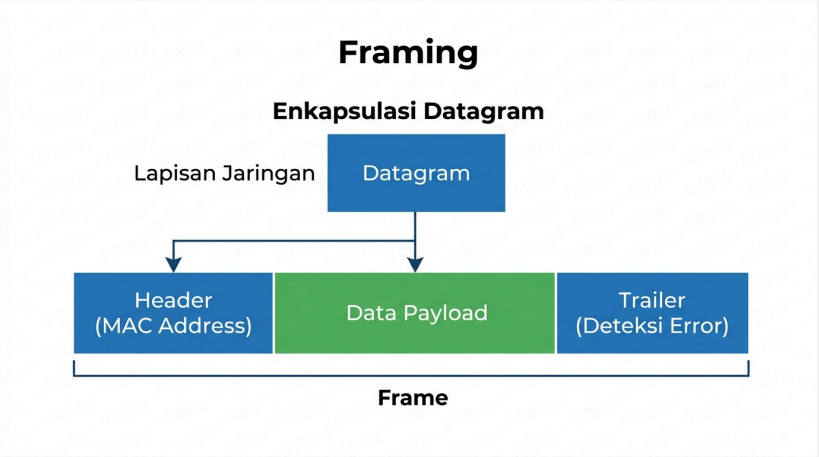
Datagram melewati beberapa link berbeda

1. Layanan Lapisan Data Link

Layanan Data Link: Framing

Framing

- Mengemas datagram dari Network Layer ke dalam sebuah **frame** dengan menambahkan *header* dan *trailer*.
- Header frame berisi **MAC address** sumber dan tujuan — berbeda dengan IP address.
- Frame menandai batas awal dan akhir data yang dikirim per satu hop.

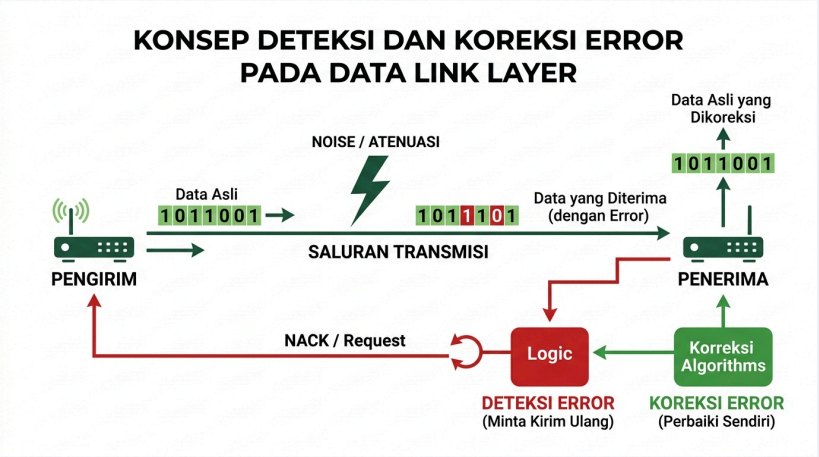


Proses enkapsulasi datagram menjadi frame

Layanan Data Link: Error Detection & Correction

Error Detection & Correction

- Error terjadi karena atenuasi sinyal dan noise selama transmisi.
- Penerima mendeteksi error, lalu meminta pengiriman ulang (*retransmission*) atau mengoreksinya sendiri.

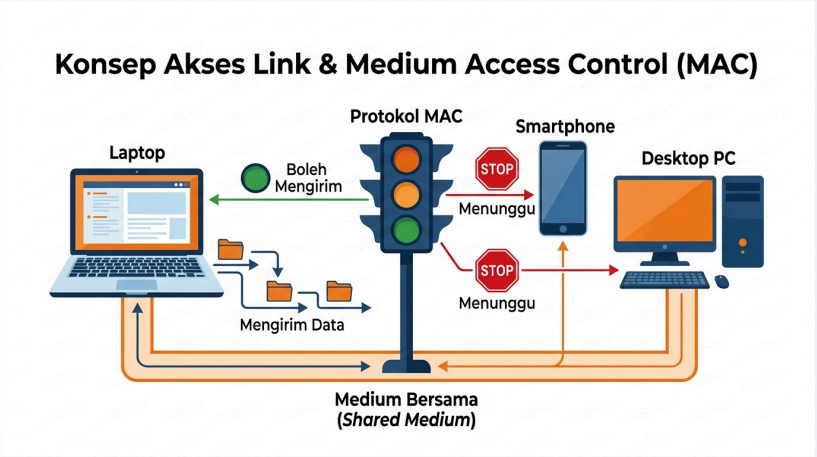


Proses penanganan error selama transmisi

Layanan Data Link: Link Access

Link Access

- Mengatur **siapa yang boleh mengirim** pada medium bersama (*shared medium*) pada satu waktu tertentu.
- Protokol yang mengatur hal ini disebut **Medium Access Control (MAC)**.



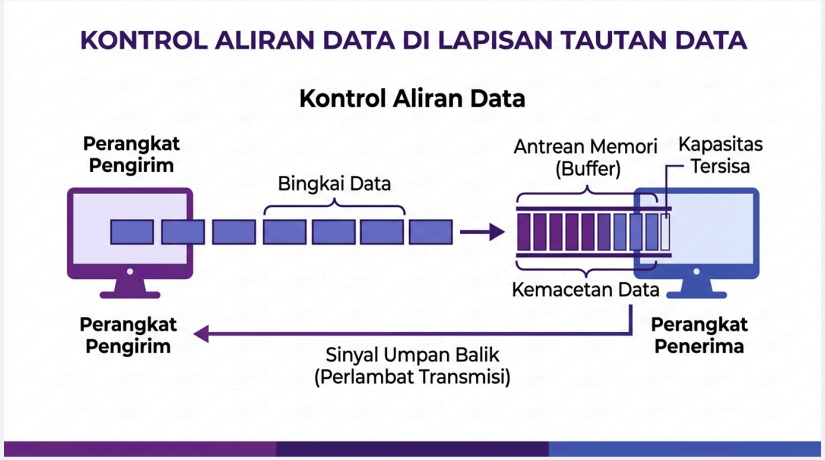
Protokol MAC mengatur akses pengiriman data

Layanan Data Link: Flow Control

Flow Control (Kontrol Aliran)

- Mengendalikan laju transmisi data secara dinamis antara perangkat pengirim dan penerima yang terhubung secara langsung.
- Mencegah terjadinya penumpukan data (kelebihan kapasitas memori antrean) apabila pengirim mengirimkan data lebih cepat daripada kemampuan penerima dalam memprosesnya.

💡 Mekanisme kontrol aliran ini diimplementasikan secara langsung pada perangkat keras Kartu Jaringan (Network Interface Card / NIC) yang terdapat pada laptop, ponsel cerdas, dan *router* Anda.



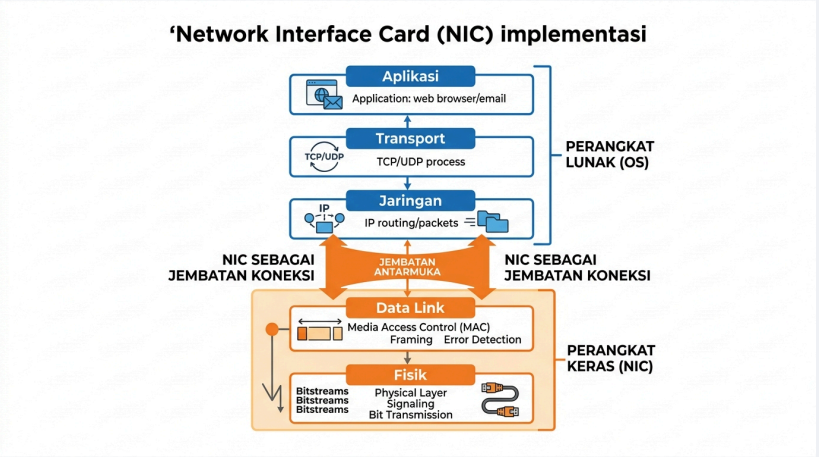
Sinyal kontrol untuk mencegah buffer penerima penuh

Implementasi: Di Mana Data Link Berada? (Part 1)

Lapisan Data Link diimplementasikan pada setiap host dan perangkat jaringan, bukan hanya di server:

- **Network Interface Card (NIC)** — chip di motherboard atau kartu PCI-e terpisah yang mengimplementasikan lapisan link dan fisik.
- NIC menghubungkan host ke link yang memungkinkan pengiriman/penerimaan frame.

> 🔌 NIC adalah "jembatan" antara dunia software (IP, TCP) dan dunia hardware (kabel, sinyal listrik).



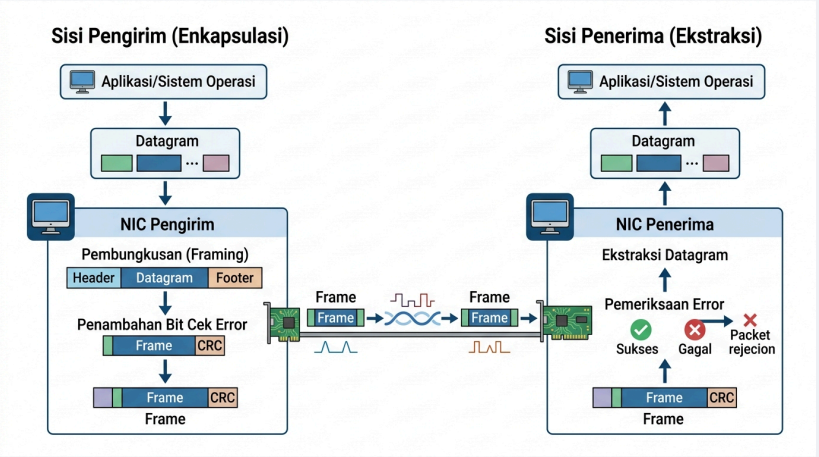
NIC mengimplementasikan lapisan Link & Physical

Implementasi: Di Mana Data Link Berada? (Part 2)

Lapisan Data Link diimplementasikan pada setiap host dan perangkat jaringan, bukan hanya di server:

- Pada sisi pengirim: mengenkapsulasi datagram ke frame, menambahkan bit error-check.
- Pada sisi penerima: memeriksa error, mengekstrak datagram, memberikannya ke network layer.

> 🔌 NIC adalah "jembatan" antara dunia software (IP, TCP) dan dunia hardware (kabel, sinyal listrik).



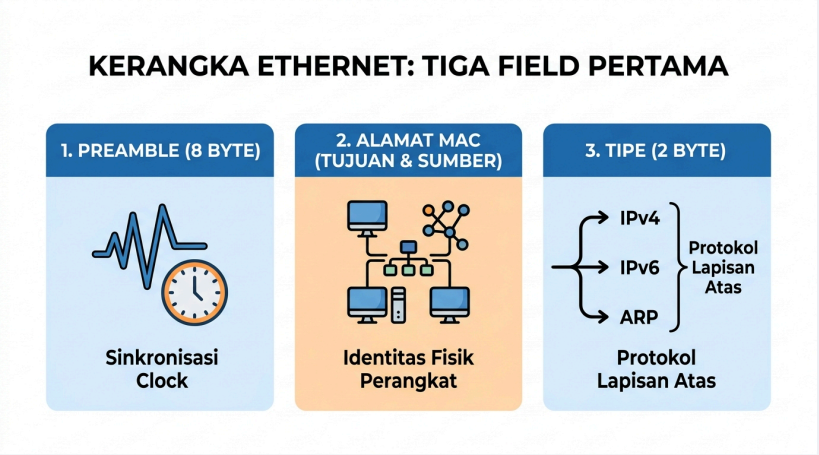
Peran aktif NIC saat mengirim dan menerima data

2. Framing & Struktur Frame

Struktur Frame Ethernet (Part 1)

Preamble 8 byte	Dest. MAC 6 byte	Source MAC 6 byte	Type 2 byte	Data (Payload) 46–1500 byte	CRC 4 byte

- **Preamble (8B):** 7 byte pola 10101010 + 1 byte 10101011. Digunakan untuk *sinkronisasi clock* antara pengirim dan penerima.
- **Dest. & Src. MAC (6B each):** Alamat fisik hardware tujuan dan sumber (48-bit).
- **Type (2B):** Menunjukkan protokol di layer atas — 0x0800 = IPv4, 0x0806 = ARP, 0x86DD = IPv6.



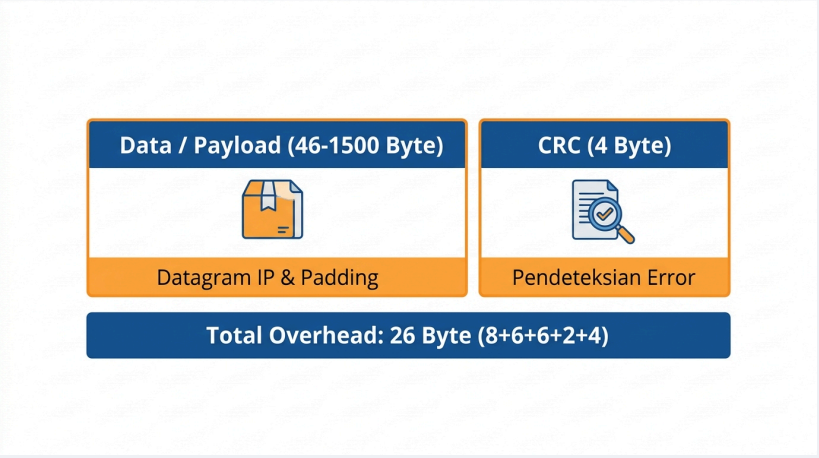
Fungsi Preamble, Alamat MAC, dan Tipe

Struktur Frame Ethernet (Part 2)

Preamble 8 byte	Dest. MAC 6 byte	Source MAC 6 byte	Type 2 byte	Data (Payload) 46–1500 byte	CRC 4 byte

- **Data / Payload (46–1500B):** Berisi datagram IP yang dienkapsulasi. Minimum 46B — jika kurang, ditambah *padding*. Maksimum 1500B (MTU Ethernet).
- **CRC (4B):** Cyclic Redundancy Check — nilai hash dari isi frame untuk mendeteksi error transmisi. Jika CRC tidak cocok, frame dibuang.

> Total overhead frame: 8+6+6+2+4 = **26 byte**. Data murni maksimal 1500/1526 ≈ 98.3% efisiensi.



Fungsi Data/Payload dan CRC

3. Deteksi Error

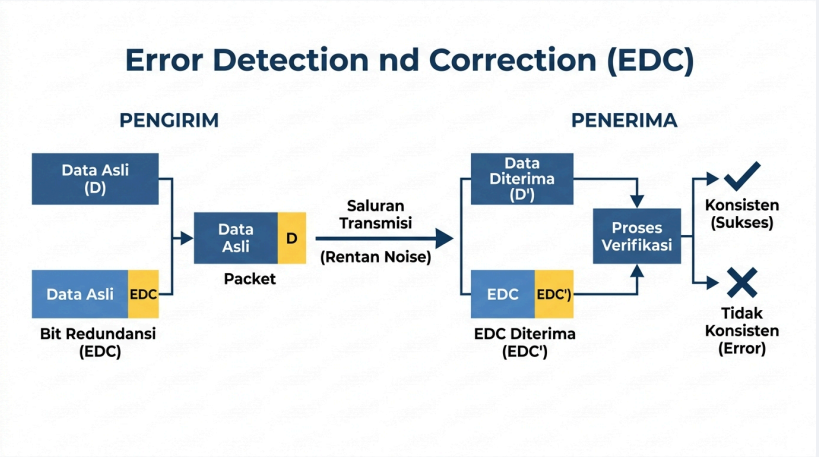
Konsep Error Detection & Correction (EDC)

Bit error dapat terjadi selama transmisi karena atenuasi, noise, dan interferensi.

D = Data yang dilindungi (termasuk header)
EDC = Bit error-detection & correction (redundansi)

Pengirim kirim: **D + EDC**
Penerima terima: **D' + EDC'** (mungkin berubah)
Penerima cek: apakah D' konsisten dengan EDC'?

⚠️ Error detection **tidak 100% reliable**! Ada kemungkinan error tidak terdeteksi (undetected error) — probabilitasnya tergantung teknik yang dipakai.



Proses logika verifikasi error pada data

Teknik 1: Parity Checking

Single Bit Parity

Menambahkan **1 bit paritas** sehingga total bit 1 dalam data menjadi genap (*even parity*) atau ganjil (*odd parity*).

```
Data: 0111000110101011
Paritas: 1 (ada 9 bit '1' → tambah 1 agar genap)
Dikirim: 01110001101010111
```

- ✅ Deteksi error **ganjil (1, 3, 5, ...)** bit yang berubah.
- ❌ Tidak bisa deteksi error genap (2, 4, ... bit berubah sekaligus).

Two-Dimensional Parity

Data disusun dalam matriks baris-kolom. Bit paritas dihitung per baris *dan* per kolom.

```
1 0 1 1 0 | 1
0 1 0 1 1 | 1
1 1 0 1 0 | 1
0 0 1 1 1 | 1
```

- ✅ Bisa mendeteksi **dan mengoreksi** single-bit error (tahu baris & kolom yang salah).
- ✅ Mendeteksi semua kombinasi 2-bit error.

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

2. Sisi Penerima (Receiver)

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

Katakanlah kita ingin mengirim karakter **'A'**. Dalam kode ASCII (7-bit), nilainya adalah: `1000001`

2. Sisi Penerima (Receiver)

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

Katakanlah kita ingin mengirim karakter **'A'**. Dalam kode ASCII (7-bit), nilainya adalah: `1000001`

- Kita menggunakan aturan **Even Parity** (jumlah total bit '1' harus genap).
- Hitung jumlah bit '1' pada data asli: Ada **dua** buah bit '1' (sudah genap).
- Maka, Bit Paritas yang ditambahkan adalah: **0**.

2. Sisi Penerima (Receiver)

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

Katakanlah kita ingin mengirim karakter **'A'**. Dalam kode ASCII (7-bit), nilainya adalah: `1000001`

- Kita menggunakan aturan **Even Parity** (jumlah total bit '1' harus genap).
- Hitung jumlah bit '1' pada data asli: Ada **dua** buah bit '1' (sudah genap).
- Maka, Bit Paritas yang ditambahkan adalah: **0**.

```
Dikirim = 1000001 0
```

2. Sisi Penerima (Receiver)

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

Katakanlah kita ingin mengirim karakter **'A'**. Dalam kode ASCII (7-bit), nilainya adalah: `1000001`

- Kita menggunakan aturan **Even Parity** (jumlah total bit '1' harus genap).
- Hitung jumlah bit '1' pada data asli: Ada **dua** buah bit '1' (sudah genap).
- Maka, Bit Paritas yang ditambahkan adalah: **0**.

```
Dikirim = 1000001 0
```

2. Sisi Penerima (Receiver)

Misalkan terjadi *noise* saat transmisi sehingga bit paling kiri berubah dari 1 menjadi 0. Data yang diterima: `0000001 0`

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

Katakanlah kita ingin mengirim karakter **'A'**. Dalam kode ASCII (7-bit), nilainya adalah: `1000001`

- Kita menggunakan aturan **Even Parity** (jumlah total bit '1' harus genap).
- Hitung jumlah bit '1' pada data asli: Ada **dua** buah bit '1' (sudah genap).
- Maka, Bit Paritas yang ditambahkan adalah: **0**.

```
Dikirim = 1000001 0
```

2. Sisi Penerima (Receiver)

Misalkan terjadi *noise* saat transmisi sehingga bit paling kiri berubah dari 1 menjadi 0. Data yang diterima: `0000001 0`

- Penerima mengetahui aturan **Even Parity**.
- Penerima menghitung ulang total bit '1' pada bingkai yang diterima.
- Jumlah bit '1' sekarang = **Satu** (bernilai ganjil).

Studi Kasus: Single Bit Parity (Even Parity)

1. Sisi Pengirim (Sender)

Katakanlah kita ingin mengirim karakter **'A'**. Dalam kode ASCII (7-bit), nilainya adalah: `1000001`

- Kita menggunakan aturan **Even Parity** (jumlah total bit '1' harus genap).
- Hitung jumlah bit '1' pada data asli: Ada **dua** buah bit '1' (sudah genap).
- Maka, Bit Paritas yang ditambahkan adalah: **0**.

```
Dikirim = 1000001 0
```

2. Sisi Penerima (Receiver)

Misalkan terjadi *noise* saat transmisi sehingga bit paling kiri berubah dari 1 menjadi 0. Data yang diterima: `0000001 0`

- Penerima mengetahui aturan **Even Parity**.
- Penerima menghitung ulang total bit '1' pada bingkai yang diterima.
- Jumlah bit '1' sekarang = **Satu** (bernilai ganjil).

❌ **ERROR TERDETEKSI!** Karena aturan mensyaratkan jumlah genap namun hasilnya ganjil, penerima tahu bahwa data telah rusak dan akan membuangnya.

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

2. Penerima (Deteksi & Koreksi)

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

2. Penerima (Deteksi & Koreksi)

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

```
(Data)    | Baris
  1 0 1 1 0 | 1
  0 1 0 1 1 | 1
  1 1 0 1 0 | 1
 -----------+---
  0 0 1 1 1 | 1
```

2. Penerima (Deteksi & Koreksi)

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

```
(Data)    | Baris
  1 0 1 1 0 | 1
  0 1 0 1 1 | 1
  1 1 0 1 0 | 1
 -----------+---
  0 0 1 1 1 | 1
```

- Orange: Bit Paritas per Baris.
- Biru: Bit Paritas per Kolom.
- Ungu: Paritas dari baris & kolom paritas.

2. Penerima (Deteksi & Koreksi)

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

```
(Data)    | Baris
  1 0 1 1 0 | 1
  0 1 0 1 1 | 1
  1 1 0 1 0 | 1
 -----------+---
  0 0 1 1 1 | 1
```

- Orange: Bit Paritas per Baris.
- Biru: Bit Paritas per Kolom.
- Ungu: Paritas dari baris & kolom paritas.

2. Penerima (Deteksi & Koreksi)

Terjadi *noise*! Bit pada **Baris 2, Kolom 2** berubah menjadi `0`.

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

```
(Data)    | Baris
  1 0 1 1 0 | 1
  0 1 0 1 1 | 1
  1 1 0 1 0 | 1
 -----------+---
  0 0 1 1 1 | 1
```

- Orange: Bit Paritas per Baris.
- Biru: Bit Paritas per Kolom.
- Ungu: Paritas dari baris & kolom paritas.

2. Penerima (Deteksi & Koreksi)

Terjadi *noise*! Bit pada **Baris 2, Kolom 2** berubah menjadi `0`.

```
 1 0 1 1 0 | 1   (Benar! → Baris)
 0 0 0 1 1 | 1   (Salah! → Baris)
 1 1 0 1 0 | 1   (Benar! → Baris)
-----------+---------------------
0 0 1 1 1 | 1  (Salah! → Kolom 2)
              ↑
        (Salah! → Kolom)
```

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

```
(Data)    | Baris
  1 0 1 1 0 | 1
  0 1 0 1 1 | 1
  1 1 0 1 0 | 1
 -----------+---
  0 0 1 1 1 | 1
```

- Orange: Bit Paritas per Baris.
- Biru: Bit Paritas per Kolom.
- Ungu: Paritas dari baris & kolom paritas.

2. Penerima (Deteksi & Koreksi)

Terjadi *noise*! Bit pada **Baris 2, Kolom 2** berubah menjadi `0`.

```
 1 0 1 1 0 | 1   (Benar! → Baris)
 0 0 0 1 1 | 1   (Salah! → Baris)
 1 1 0 1 0 | 1   (Benar! → Baris)
-----------+---------------------
0 0 1 1 1 | 1  (Salah! → Kolom 2)
              ↑
        (Salah! → Kolom)
```

- Baris ke-2 terdeteksi ganjil ❌
- Kolom ke-2 terdeteksi ganjil ❌

Studi Kasus: Two-Dimensional Parity (Koreksi)

1. Pengirim (Menyusun Matriks)

Data asli dibagi menjadi matriks. Hitung paritas genap (*even parity*) untuk tiap baris dan tiap kolom.

```
(Data)    | Baris
  1 0 1 1 0 | 1
  0 1 0 1 1 | 1
  1 1 0 1 0 | 1
 -----------+---
  0 0 1 1 1 | 1
```

- Orange: Bit Paritas per Baris.
- Biru: Bit Paritas per Kolom.
- Ungu: Paritas dari baris & kolom paritas.

2. Penerima (Deteksi & Koreksi)

Terjadi *noise*! Bit pada **Baris 2, Kolom 2** berubah menjadi `0`.

```
 1 0 1 1 0 | 1   (Benar! → Baris)
 0 0 0 1 1 | 1   (Salah! → Baris)
 1 1 0 1 0 | 1   (Benar! → Baris)
-----------+---------------------
0 0 1 1 1 | 1  (Salah! → Kolom 2)
              ↑
        (Salah! → Kolom)
```

- Baris ke-2 terdeteksi ganjil ❌
- Kolom ke-2 terdeteksi ganjil ❌

> ✅ **KOREKSI BERHASIL!** Persilangan baris ke-2 dan kolom ke-2 menunjukkan letak *error* secara pasti. Penerima cukup mengembalikan `0` menjadi `1`.

Teknik 2: Cyclic Redundancy Check (CRC)

Prinsip CRC

- **D** = data bits (sebagai bilangan biner).
- **G** = generator polynomial (r+1 bit, disepakati pengirim & penerima).
- Pengirim menghitung **R** (r bit) sedemikian sehingga:
  `D·2ʳ XOR R` habis dibagi G tanpa sisa.
- Penerima membagi `D'·2ʳ XOR R'` dengan G. Jika sisa ≠ 0 → error terdeteksi.

Contoh CRC

```
D = 101110 (data, 6 bit)
G = 1001 (generator, 4 bit → r=3)

D·2³ = 101110000
Bagi dengan G (long division XOR):
R = 011

Dikirim: 101110011
```

> ✅ CRC digunakan di **Ethernet, Wi-Fi, USB, Bluetooth** karena kemampuannya mendeteksi burst error.

CRC 32-bit (CRC-32) yang dipakai Ethernet mampu mendeteksi semua burst error ≤ 32 bit, dan sebagian besar burst error yang lebih panjang.

Contoh Penerapan CRC

Di Sisi Pengirim

D: 101110 | **G:** 1001 (r=3)

1. Tambah r bit 0: 101110000

2. Bagi (XOR) dengan G:

```
                _______________
  1001   / 1 0 1 1 1 0 0 0 0
           1 0 0 1
           -------
             0 1 0 1
               0 0 0 0
               -------
               1 0 1 0
               1 0 0 1
               -------
                 0 1 1 0
                 0 0 0 0
                 -------
                   1 1 0 0
                   1 0 0 1
                   -------
                     1 0 1 0
                     1 0 0 1
                     -------
                       0 1 1 (R)
```

3. Dikirim: D + R = 101110011

Di Sisi Penerima

Diterima: 101110011 | **G:** 1001

1. Bagi (XOR) dengan G:

```
                _______________
  1001   / 1 0 1 1 1 0 0 1 1
           1 0 0 1
           -------
             0 1 0 1
               0 0 0 0
               -------
               1 0 1 0
               1 0 0 1
               -------
                 0 1 1 0
                 0 0 0 0
                 -------
                   1 1 0 1
                   1 0 0 1
                   -------
                     1 0 0 1
                     1 0 0 1
                     -------
                       0 0 0 (Sisa)
```

2. Cek Sisa: Sisa = 000 → **TIDAK ADA ERROR.**

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

1. Anggap isi segmen UDP/TCP (termasuk header) sebagai deretan **integer 16-bit**.

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

1. Anggap isi segmen UDP/TCP (termasuk header) sebagai deretan **integer 16-bit**.
2. Jumlahkan semua integer tersebut menggunakan **aritmatika 1's complement**.

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

1. Anggap isi segmen UDP/TCP (termasuk header) sebagai deretan **integer 16-bit**.
2. Jumlahkan semua integer tersebut menggunakan **aritmatika 1's complement**.
3. Hasil penjumlahan di-**complement** (balik semua bit) → ini adalah **checksum**.

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

1. Anggap isi segmen UDP/TCP (termasuk header) sebagai deretan **integer 16-bit**.
2. Jumlahkan semua integer tersebut menggunakan **aritmatika 1's complement**.
3. Hasil penjumlahan di-**complement** (balik semua bit) → ini adalah **checksum**.
4. Sisipkan nilai checksum ke dalam field checksum di header.

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

1. Anggap isi segmen UDP/TCP (termasuk header) sebagai deretan **integer 16-bit**.
2. Jumlahkan semua integer tersebut menggunakan **aritmatika 1's complement**.
3. Hasil penjumlahan di-**complement** (balik semua bit) → ini adalah **checksum**.
4. Sisipkan nilai checksum ke dalam field checksum di header.

> 💡 Checksum digunakan di **UDP & TCP header**, serta **IP header**. Kekuatannya lebih lemah dari CRC, tapi sangat cepat dihitung oleh *software*.

Teknik 3: Internet Checksum (Pengirim)

Cara Kerja (Pengirim)

1. Anggap isi segmen UDP/TCP (termasuk header) sebagai deretan **integer 16-bit**.
2. Jumlahkan semua integer tersebut menggunakan **aritmatika 1's complement**.
3. Hasil penjumlahan di-**complement** (balik semua bit) → ini adalah **checksum**.
4. Sisipkan nilai checksum ke dalam field checksum di header.

> 💡 Checksum digunakan di **UDP & TCP header**, serta **IP header**. Kekuatannya lebih lemah dari CRC, tapi sangat cepat dihitung oleh *software*.

Contoh Penjumlahan (Pengirim)

```
0110011001100110
+ 0101010101010101
------------------
1011101110110111
← complement →
0100010001001000 = checksum
```

Teknik 3: Internet Checksum (Penerima)

Cara Kerja (Penerima)

Teknik 3: Internet Checksum (Penerima)

Cara Kerja (Penerima)

1. Jumlahkan semua integer 16-bit yang diterima *termasuk* field checksum yang telah disisipkan tadi.

Teknik 3: Internet Checksum (Penerima)

Cara Kerja (Penerima)

1. Jumlahkan semua integer 16-bit yang diterima *termasuk* field checksum yang telah disisipkan tadi.
2. Jika hasil penjumlahannya = `1111...1111` (semua bit 1) → **tidak ada error**.

Teknik 3: Internet Checksum (Penerima)

Cara Kerja (Penerima)

1. Jumlahkan semua integer 16-bit yang diterima *termasuk* field checksum yang telah disisipkan tadi.
2. Jika hasil penjumlahannya = `1111...1111` (semua bit 1) → **tidak ada error**.
3. Jika ada satu saja bit yang bernilai 0 → **error terdeteksi**, segmen tersebut langsung dibuang.

Teknik 3: Internet Checksum (Penerima)

Cara Kerja (Penerima)

1. Jumlahkan semua integer 16-bit yang diterima *termasuk* field checksum yang telah disisipkan tadi.
2. Jika hasil penjumlahannya = `1111...1111` (semua bit 1) → **tidak ada error**.
3. Jika ada satu saja bit yang bernilai 0 → **error terdeteksi**, segmen tersebut langsung dibuang.

Contoh Verifikasi (Penerima)

```
1011101110110111 (Hasil Jumlah)
+ 0100010001001000 (Checksum)
------------------
1111111111111111
← Valid! →
```

4. Multiple Access Protocols

Point-to-Point vs Broadcast Link

Point-to-Point Link

- Hanya **dua node** yang terhubung langsung di satu link.
- Tidak ada masalah "tabrakan" — channel eksklusif.
- Contoh: Kabel Ethernet antara PC dan switch, PPP (dial-up modem).

Broadcast Link (Shared Medium)

- **Banyak node** berbagi satu channel yang sama secara bersamaan.
- Jika dua node kirim data serentak → **Collision (tabrakan)** → data rusak, tidak terbaca.
- Contoh: Wi-Fi (WLAN), Ethernet bus lama (coaxial), Bluetooth.

> ⚠️ Collision = sinyal dari 2+ sumber bercampur dan tidak dapat dibedakan. Butuh protokol MAC untuk mengatur!

MAC Protocol menentukan cara node bergantian menggunakan shared channel sehingga collision dapat dihindari atau pulih dari collision dengan cepat.

Tiga Kelas Protokol MAC

📅 Channel Partitioning

Bagi channel menjadi irisan kecil (time slot, frekuensi, kode). Alokasikan irisan secara eksklusif ke tiap node.

- TDMA (Time Division)
- FDMA (Frequency Division)
- CDMA (Code Division)

✅ Tanpa collision
❌ Wasteful saat node sepi

🎲 Random Access

Node kirim kapan pun mau. Jika collision terjadi, ada mekanisme deteksi & retransmisi dengan waktu acak.

- ALOHA (Slotted & Pure)
- CSMA / CSMA-CD
- CSMA-CA (Wi-Fi)

✅ Efisien saat beban ringan
❌ Collision bisa terjadi

🎤 Taking Turns

Node bergiliran secara teratur (polling atau token). Tidak ada collision, tapi ada overhead koordinasi.

- Polling (master-slave)
- Token Ring / Token Bus

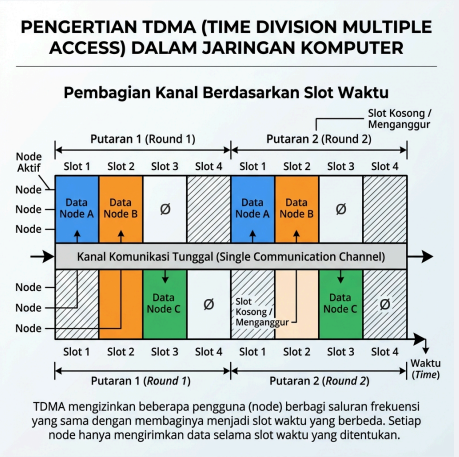
✅ Efisiensi tinggi
❌ Single point of failure (token/master)

Channel Partitioning: TDMA

TDMA (Time Division Multiple Access)

Kapasitas saluran transmisi (*channel*) tunggal dibagi berdasarkan **waktu (time slot)**.

- Tiap *node* mendapat jatah slot waktu yang tetap pada setiap putaran (*round*).
- *Node* yang sedang tidak memiliki data akan tetap "membuang" slot waktunya (inefisien saat lalu lintas sepi).
- **Contoh Implementasi:** Jaringan seluler 2G (GSM).



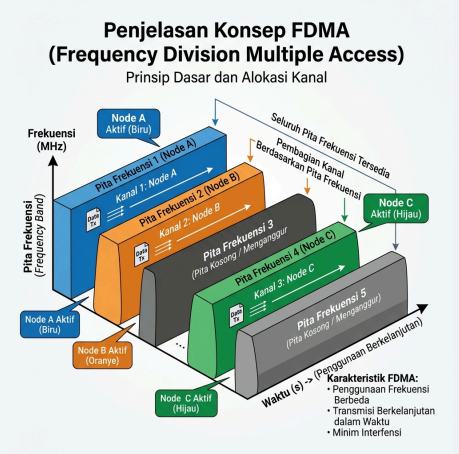
Transmisi data diatur ke dalam slot waktu berurutan

Channel Partitioning: FDMA

FDMA (Frequency Division Multiple Access)

Kapasitas saluran transmisi dibagi berdasarkan **pita frekuensi (frequency band)**.

- Tiap *node* dialokasikan pita frekuensi eksklusif secara permanen.
- Pita frekuensi dari *node* yang sedang tidak mentransmisikan data akan terbuang percuma (menjadi pita kosong/menganggur).
- **Contoh Implementasi:** Siaran Radio FM analog, jaringan seluler 1G (AMPS).



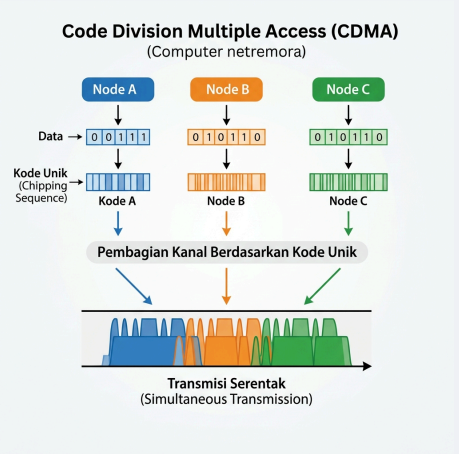
Transmisi data diatur ke dalam pita frekuensi terpisah

Channel Partitioning: CDMA

CDMA (Code Division Multiple Access)

Kapasitas saluran tidak dibagi berdasarkan waktu atau frekuensi, melainkan berdasarkan **kode unik (chipping sequence)**.

- Setiap *node* diberikan sebuah kode matematika unik yang saling ortogonal (tidak mengganggu satu sama lain).
- Semua *node* dapat mentransmisikan data **secara bersamaan serentak** pada **frekuensi yang sama**.
- Penerima menggunakan operasi matematika (*dot product*) dengan kode yang sama untuk mengekstrak data murni dari pengirim tertentu di antara sinyal-sinyal yang tercampur.
- **Contoh Implementasi:** Jaringan seluler 3G (WCDMA), komunikasi satelit, dan militer.



Transmisi dicampur dengan kode unik (Chipping Sequence)

Random Access: Slotted ALOHA

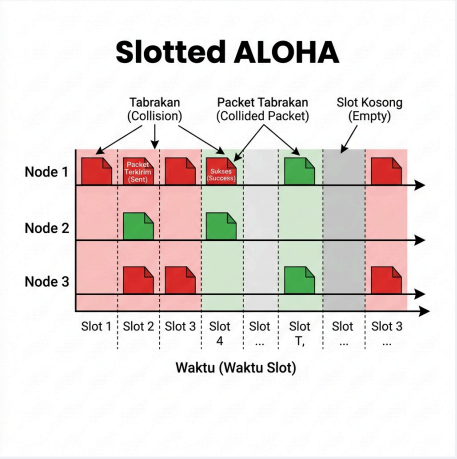
Asumsi & Cara Kerja

- Semua frame berukuran sama.
- Waktu dibagi ke dalam **slot** berukuran tetap (= waktu kirim 1 frame).
- Node hanya boleh mulai kirim di awal slot.
- Jika **collision**: node retransmisi di slot berikutnya dengan **probabilitas p**.

Efisiensi Maksimum

Dengan N node dan probabilitas kirim p per slot, efisiensi optimal mendekati **1/e ≈ 37%** saat N sangat besar.

> ✅ Sederhana, node aktif tunggal bisa pakai penuh channel.
> ❌ Efisiensi max hanya 37% — sangat boros!



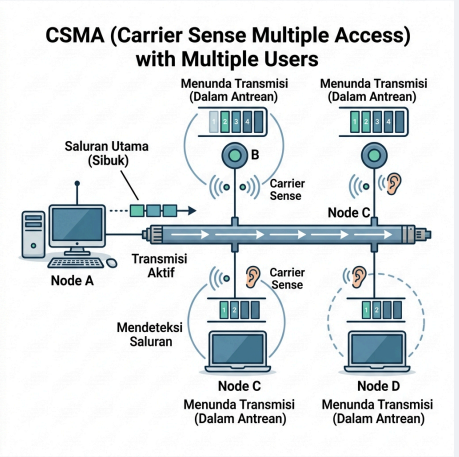
Transmisi data pada tiap node berdasarkan slot waktu tertentu

Protokol Ethernet - CSMA

CSMA (Carrier Sense Multiple Access)

"Dengarkan dulu sebelum bicara"

- Sebelum kirim, node *mendengarkan* (sense) apakah channel sedang kosong.
- Jika **idle**: kirim frame sepenuhnya.
- Jika **busy**: tunda, tunggu sampai idle lalu kirim.
- ❌ Masih bisa collision! (karena *propagation delay* — sinyal butuh waktu merambat).



Prinsip dasar 'Listen Before Talk' pada saluran jaringan

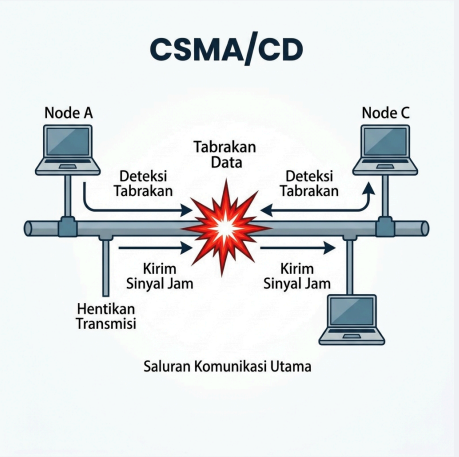
Protokol Ethernet - CSMA/CD

CSMA/CD (Collision Detection)

"Dengarkan sambil bicara, hentikan jika tabrakan"

- Selama mengirim, node terus *mendengarkan* apakah ada sinyal lain (collision).
- Jika collision terdeteksi: **hentikan transmisi segera**, kirim *jam signal* untuk memberi tahu semua node.
- Tunggu waktu **random (exponential backoff)** lalu coba kirim lagi.
- ✅ Mengurangi waktu buang — tidak perlu menunggu frame selesai sebelum tahu ada collision.

> CSMA/CD digunakan di **Ethernet kabel**. Wi-Fi menggunakan CSMA/CA (Collision Avoidance) karena tidak bisa mendeteksi collision saat kirim.



Proses deteksi tabrakan data dan pengiriman Sinyal Jam

Exponential Backoff — Mekanisme Coba Ulang

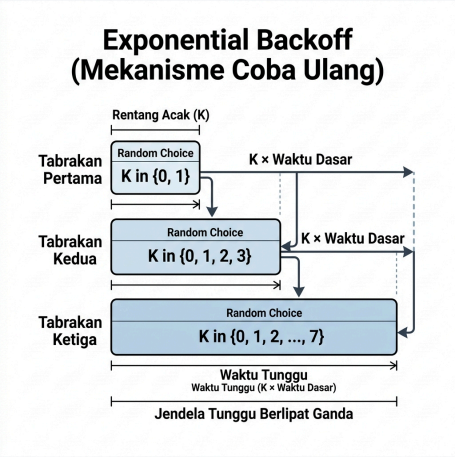
Setelah collision ke-n, node menunggu waktu acak yang dipilih dari rentang:

```
Tunggu: K × 512 bit-time

K dipilih acak dari {0, 1, ..., 2ⁿ-1}

Collision ke-1: K ∈ {0, 1}
Collision ke-2: K ∈ {0, 1, 2, 3}
Collision ke-3: K ∈ {0, ..., 7}
...
Collision ke-10: K ∈ {0, ..., 1023}
Collision ke-16: Give up! ERROR
```

> 💡 Semakin sering collision, semakin lama menunggu — ini *desentralisasi beban* secara otomatis. Mirip antrian kasir: jika penuh, tunda sebentar, coba lagi nanti.



Jendela rentang waktu tunggu menjadi dua kali lipat pasca tabrakan

5. MAC Address & ARP

MAC Address: Identitas Fisik Hardware

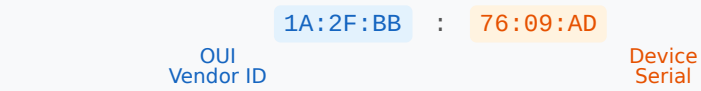
Setiap antarmuka jaringan (NIC) memiliki **MAC address 48-bit (6 byte)** yang unik secara global, ditulis dalam notasi heksadesimal:

`1A:2F:BB:76:09:AD`

- **3 byte pertama** = OUI (Organizationally Unique Identifier) — mengidentifikasi *produsen* NIC. Dialokasikan oleh IEEE.
- **3 byte terakhir** = Nomor unik yang ditetapkan produsen untuk tiap unit.
- Tersimpan di ROM pada chip NIC — biasanya tidak bisa diubah (namun bisa *spoofed* via software).

> 🆚 **MAC vs IP:** MAC = alamat *fisik permanen* yang mengidentifikasi hardware NIC. IP = alamat *logis dinamis* yang bergantung pada lokasi jaringan. MAC seperti NIK, IP seperti alamat rumah.

Struktur MAC Address



Contoh OUI terkenal:

- 00:50:56 = VMware
- A4:C3:F0 = Intel
- DC:A6:32 = Raspberry Pi

3 byte OUI + 3 byte serial = 48-bit MAC address

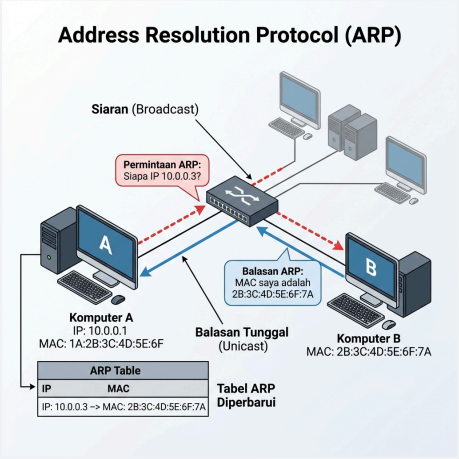
ARP: Address Resolution Protocol

Masalah: Network layer punya IP address tujuan, tapi Data Link butuh MAC address. Bagaimana caranya?

Jawaban: ARP (RFC 826)

1. **ARP Table**: Tiap node menyimpan tabel IP→MAC dengan TTL (biasanya 20 menit).
2. **ARP Query**: Jika MAC tidak ada di tabel, node broadcast ARP request ke seluruh LAN:
   "Siapa yang punya IP 192.168.1.5? Beritahu MAC-mu!"
3. **ARP Reply**: Node yang memiliki IP tersebut membalas secara unicast dengan MAC address-nya.
4. Pengirim menyimpan pasangan IP→MAC ke tabel, lalu kirim frame.

> ⚠️ ARP hanya bekerja di dalam **satu subnet** (satu LAN). Untuk kirim ke subnet lain, frame dikirim ke **gateway (router)** menggunakan MAC address router.



Proses resolusi ARP dalam satu LAN

Pengiriman Antar Subnet via Router

Ketika host A (subnet 111.111.111.0) ingin kirim ke host B (subnet 222.222.222.0):

Langkah 1: A → Router

- A tahu IP tujuan = 222.222.222.1 (berbeda subnet).
- A kirim frame ke **MAC address Router** (via ARP).
- IP Dst dalam datagram = IP B (tetap).
- MAC Dst dalam frame = MAC Router (berubah per hop).

Langkah 2: Router Proses

- Router terima frame, ekstrak datagram.
- Lihat IP Dst = 222.222.222.1.
- Tentukan interface keluar lewat **routing table**.
- ARP untuk mendapatkan **MAC B**.

Langkah 3: Router → B

- Router buat frame baru dengan **MAC Src = Router**, **MAC Dst = MAC B**.
- IP Dst dalam datagram tetap = IP B.
- B terima frame, ambil datagram.

🔑 **Kunci penting:** IP address tujuan *tidak berubah* selama perjalanan. Tapi MAC address *berubah di setiap hop*. IP = alamat global, MAC = alamat lokal per link.

6. Ethernet

Ethernet: Teknologi LAN Dominan

- Pertama kali dikembangkan oleh **Xerox PARC** tahun 1973 (Robert Metcalfe).
- Standar terbuka (IEEE 802.3) — satu chip bisa mendukung banyak kecepatan.
- Evolusi kecepatan: **10 Mbps → 100 Mbps → 1 Gbps → 10 Gbps → 400 Gbps**.
- Bersifat **connectionless**: tidak ada handshaking sebelum kirim frame.
- Bersifat **unreliable**: penerima tidak mengirim ACK/NAK. Jika CRC error, frame dibuang begitu saja — recovery diserahkan ke TCP di atasnya.

> Alasan Ethernet bertahan selama 50+ tahun: **sederhana, murah, dan berhasil mengikuti perlombaan kecepatan** tanpa mengubah antarmuka pemrogramannya.

Evolusi Topologi Ethernet

Bus (1970–1990-an):
Semua host berbagi 1 kabel coaxial. Jika ada yang kirim, semua "dengar". Satu domain collision besar.

Switched (1990-an–sekarang):
Host terhubung ke switch via kabel UTP dedicated. Tiap port = domain collision sendiri. Tidak ada collision!

Dari topologi bus ke switched

Standar Ethernet (IEEE 802.3)

Standar	Kecepatan	Medium	Jarak Maks	Keterangan
10BASE-T	10 Mbps	UTP Cat 3	100 m	Ethernet awal, sudah usang
100BASE-TX	100 Mbps	UTP Cat 5	100 m	Fast Ethernet, masih ada di gedung lama
1000BASE-T	**1 Gbps**	UTP Cat 5e/6	100 m	Gigabit Ethernet — standar umum saat ini
10GBASE-T	10 Gbps	UTP Cat 6a/7	100 m	Data Center, perkantoran high-end
1000BASE-LX	1 Gbps	Single-mode fiber	5–10 km	Backbone kampus / gedung
400GBASE	400 Gbps	Multi-fiber	~100 m	Hyperscale Data Center (cloud)

Notasi: **Kecepatan BASE Medium** — contoh "1000BASE-T" = 1000 Mbps (1 Gbps), baseband, medium twisted pair.

7. Ethernet Switch

Ethernet Switch: Perangkat Layer 2

Cara Kerja Switch

- Switch adalah perangkat **link-layer aktif**: memeriksa MAC address tujuan frame, lalu meneruskannya hanya ke port yang tepat.
- **Store-and-Forward:** Terima frame lengkap, periksa CRC, baru teruskan.
- Setiap port switch = domain collision tersendiri → tidak ada CSMA/CD yang diperlukan per port.
- Mendukung **transmisi simultan**: A↔B dan C↔D bisa terjadi serentak tanpa interferensi.
- **Transparent:** Host tidak tahu ada switch — dari sudut pandang host, seperti terhubung langsung.

Switch vs Hub

Aspek	Hub	Switch
Layer OSI	Physical (L1)	Data Link (L2)
Kecerdasan	Broadcast semua port	Forward berdasarkan MAC
Collision Domain	Satu besar	Per port
Simultaneous	Tidak	Ya
Full-Duplex	Tidak	Ya
Harga	Murah	Lebih mahal

✅ Hub sudah hampir tidak digunakan. Switch adalah standar de-facto di semua jaringan LAN modern.

Switch: Self-Learning & Forwarding

Switch belajar secara otomatis — **tidak perlu dikonfigurasi manual**:

1. **Catat MAC sumber:** Setiap frame yang diterima, switch mencatat MAC address pengirim dan port masuknya ke *forwarding table* (SAT).
2. **Lookup MAC tujuan:**
   - Jika MAC tujuan *ada* di tabel → **forward** hanya ke port tersebut.
   - Jika MAC tujuan *tidak ada* → **flood** ke semua port kecuali port masuk.
   - Jika tujuan = pengirim (same segment) → **filter** (buang, tidak perlu diteruskan).
3. **TTL entry:** Setiap entri punya TTL. Jika tidak ada aktivitas, entri dihapus.

> 💡 Inilah mengapa switch "plug-and-play" — cukup colokkan kabel, switch akan belajar sendiri dalam hitungan detik.

Switch Forwarding Table (Contoh)

MAC Address	Interface	TTL
1A:2F:BB:76:09:AD	Port 1	60s
58:23:D7:FA:20:B0	Port 3	45s
0C:C4:7A:12:34:56	Port 5	30s
AA:BB:CC:DD:EE:FF	*???*	*Belum ada → Flood*

Frame dari A ke B:
• Src A → simpan ke Port 1
• Dst B ada di Port 3 → forward ke Port 3 saja ✅

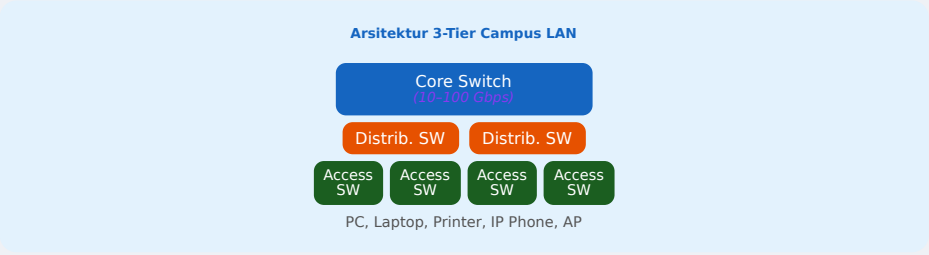
Menghubungkan Banyak Switch

Switch dapat dihubungkan satu sama lain untuk membentuk jaringan LAN yang lebih besar:

- Self-learning tetap bekerja — tabel di setiap switch dibangun secara independen.
- Jika S1 menerima frame ke tujuan yang tidak dikenalnya, S1 akan *flood* ke semua port termasuk uplink ke S2, S3, dst.
- Tabel setiap switch akan terisi seiring traffic berjalan.

🏫 Di jaringan kampus modern, arsitektur 3-tier umum digunakan: **Access → Distribution → Core** switch. Access terhubung ke end-device, Core menghubungkan antar gedung dengan kecepatan sangat tinggi (10–100 Gbps).

⚠️ Masalah loop! Tanpa mekanisme tambahan, flood bisa menyebabkan storm broadcast yang tak terhenti. Solusinya: **Spanning Tree Protocol (STP/RSTP)** — memblokir port tertentu agar tidak terbentuk loop.



Hierarki switch di jaringan kampus

Switch vs Router: Apa Bedanya?

Aspek	Switch (L2)	Router (L3)
Layer OSI	Data Link (L2)	Network (L3)
Alamat yang digunakan	MAC address	IP address
Tabel yang dibangun	Forwarding table (MAC → Port)	Routing table (IP network → Interface)
Cara belajar	Self-learning (otomatis)	Routing protocol (RIP, OSPF, BGP) atau manual
Broadcast domain	Tidak memisahkan (flood broadcast)	Memisahkan (broadcast tidak dilewatkan)
Koneksi antar jaringan	Tidak bisa (hanya dalam satu subnet)	Ya (bisa hubungkan banyak subnet)
Plug-and-play	Ya — zero config	Tidak — perlu konfigurasi IP, routing

🔌 **Gunakan Switch** untuk menghubungkan perangkat dalam satu subnet (LAN) yang sama — lebih murah dan plug-and-play.

🌐 **Gunakan Router** untuk menghubungkan subnet yang berbeda atau ke internet — butuh konfigurasi tapi lebih powerful.

Rangkuman Pertemuan 4

Lapisan Data Link & Error Handling

- Data Link: framing, link access, flow control, error detection.
- Frame Ethernet: Preamble | MAC Dst | MAC Src | Type | Data | CRC.
- Deteksi error: **Parity** (sederhana), **Checksum** (UDP/TCP/IP), **CRC** (Ethernet — paling powerful).
- Implementasi di **NIC** — ada di setiap host & perangkat jaringan.

MAC, ARP, Ethernet & Switch

- MAC address = 48-bit, unik per NIC, tidak berubah per hop.
- ARP: memetakan IP → MAC dalam satu subnet (broadcast query, unicast reply).
- Multiple Access: Channel Partition (TDMA/FDMA) · Random (CSMA/CD) · Taking Turns.
- Ethernet: connectionless, unreliable, CSMA/CD, CRC-32.
- Switch: self-learning forwarding table, per-port collision domain, simultan.
- Switch (L2/MAC) vs Router (L3/IP) — berbeda fungsi dan skala.

Tugas Pertemuan 4

Dikerjakan analitik, ditulis tangan di kertas folio bergaris, di-scan dengan jelas, lalu kirim format PDF ke e-learning.

1. Di sebuah laboratorium komputer terdapat 30 PC yang semuanya terhubung ke **satu hub** 100 Mbps tipe lama. Mahasiswa mengeluhkan jaringan sangat lambat terutama saat banyak yang menggunakan serentak. **Jelaskan secara teknis berdasarkan logika CSMA/CD** mengapa hal ini terjadi (kaitkan dengan *collision domain* dan efisiensi *bandwidth*). **Rancanglah** solusi *upgrade* menggunakan *switch* dan jelaskan peningkatan *bandwidth* efektif per PC.

2. Seorang administrator mendeteksi anomali di jaringan LAN. Ternyata terjadi serangan **ARP Spoofing** — penyerang mengirimkan *ARP Reply* palsu secara terus-menerus. **Uraikan tahap demi tahap** bagaimana serangan ini bekerja memanipulasi pemetaan tabel IP ke MAC di **Node** korban, dan **analisislah** celah kelemahan pada protokol ARP yang dimanfaatkan oleh penyerang (berdasarkan cara kerjanya di materi).

3. Sebuah *frame* Ethernet dari *Server* diterima oleh *NIC (Network Interface Card)* PC Anda, namun hasil perhitungan CRC (Cyclic Redundancy Check) di sisi penerima tidak cocok dengan nilai CRC di *trailer frame*. **Analisislah skenario ini**: (a) Tindakan pasti apa yang akan diambil oleh NIC terhadap *frame* tersebut? (b) Karena *Ethernet* bersifat *unreliable*, lapisan manakah yang bertanggung jawab meminta data dikirim ulang? Jelaskan alasannya.

Tulis tangan asli | No AI Allowed | Scan PDF ke E-learning | Deadline: H-1 PTM 5

Tanya Jawab & Diskusi

Pertemuan berikutnya: **Lapisan Transport — UDP & TCP**

Referensi mandiri: Kurose & Ross Chapter 6 (Link Layer)